

## Code-Layout, Readability and Reusability

|                      |                                                             |
|----------------------|-------------------------------------------------------------|
| <b>Date</b>          | 10 July 2026                                                |
| <b>Team ID</b>       | XXXXXX                                                      |
| <b>Project Name</b>  | OptiCrop: Smart Agricultural Production Optimization Engine |
| <b>Maximum Marks</b> | 5 Marks                                                     |

### Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

### Code Layout Checklist:

| S.No | Code Quality Parameter   | Description                                       | Followed (Yes / No / Partial) | Remarks                                                                                                                         |
|------|--------------------------|---------------------------------------------------|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| 1    | Consistent Indentation   | Uniform spacing/tabs used throughout the code     | Yes                           | Followed Python PEP 8 formatting standards                                                                                      |
| 2    | Proper File Structure    | Files and folders are logically organized         | Yes                           | Separate folders for templates, static files, dataset, and model                                                                |
| 3    | Meaningful Variable Name | Variables reflect their purpose clearly           | Yes                           | Employed specific identifiers like nitrogen, temperature, humidity, and prediction_result to enhance clarity.                   |
| 4    | Function / Method Names  | Functions are descriptively named                 | Yes                           | Functions like <code>predict_crop()</code> , <code>load_model()</code> , and <code>preprocess_data()</code> improve readability |
| 5    | Code Comments            | Inline and block comments explain important logic | Yes                           | Comments added for preprocessing, model loading, and prediction logic                                                           |
| 6    | Modular Design           | Code is split into reusable functions/modules     | Yes                           | ML model, preprocessing, and Flask routes are implemented as separate modules                                                   |
| 7    | No Redundant Code        | Duplicate or unused code is removed               | Yes                           | Reusable functions used to avoid duplication                                                                                    |
| 8    | Error Handling           | Exceptions and errors are handled properly        | Yes                           | Input validation and exception handling implemented for prediction requests                                                     |

### Reusable Components / Modules:

| S.No | Component / Module Name | Language / Technology | Where Reused | Reusability Level (High / Medium / Low) |
|------|-------------------------|-----------------------|--------------|-----------------------------------------|
|------|-------------------------|-----------------------|--------------|-----------------------------------------|

|   |                           |                 |                                    |        |
|---|---------------------------|-----------------|------------------------------------|--------|
| 1 | Data Preprocessing Module | Python, Pandas  | Model training and Prediction      | High   |
| 2 | Machine Learning Model    | Sckit-learn     | Flask Prediction module            | High   |
| 3 | Prediction Function       | Python          | Crop Recommendation endpoint       | High   |
| 4 | Flask Routes              | Flask           | Handle user requests and responses | Medium |
| 5 | HTML Templates            | HTML, Bootstrap | Input and result pages             | Medium |

#### Overall Code Quality Assessment:

| Aspect                   | Rating (1-5) | Comments                                                                                           |
|--------------------------|--------------|----------------------------------------------------------------------------------------------------|
| Code Layout & Structure  | 5            | Well-organized project structure with separate modules and folders                                 |
| Readability              | 5            | Meaningful naming conventions and consistent formatting improve readability                        |
| Reusability              | 5            | Modular functions and reusable components reduce code duplication                                  |
| Documentation / Comments | 4            | Important sections are documented with comments; additional documentation can be added if required |
| Overall Score            | 4.8          | Clean, maintainable, and reusable codebase following good software engineering practices           |